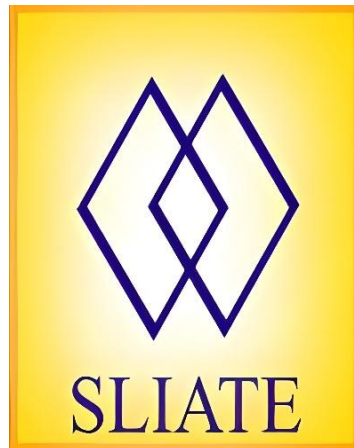


SMART IT ASSET LIFECYCLE & SECURITY COMPLIANCE MANAGEMENT SYSTEM



Student Name: W.G.C.D. Ravihara

Index Number: DEH/IT/2324/F/57

Course Code: HNDIT4052

Course Title: Programming Individual Project

Advanced Technological Institute, Dehiwala (SLIATE)

Submission Date: July 5, 2026

ACKNOWLEDGEMENT

I would like to start by thanking the Sri Lanka Institute of Advanced Technological Education (SLIATE) and the Advanced Technological Institute, Dehiwala, for giving me the academic environment and resources needed to complete this individual project.

I am sincerely grateful to my supervisor and the lecturers of the HNDIT department. Their guidance, patience, and helpful feedback throughout the planning, building, and testing stages made a real difference to the final result.

Finally, I want to thank my family and friends for their constant encouragement. Their support kept me motivated during the busy weeks of development and made it possible to finish this project.

ABSTRACT

Keeping track of computers, laptops, and other IT equipment is a real challenge for many organizations. Most still depend on spreadsheets or separate tools that do not talk to each other, which leads to missing records and slow detection of security problems such as a disabled antivirus or an outdated operating system.

This project introduces the Smart IT Asset Lifecycle & Security Compliance Management System, a web-based platform that brings asset tracking and security monitoring together in one place. It has three main parts: a small Python program (agent) that runs quietly in the background on each Windows computer, a Node.js and Express backend connected to a PostgreSQL database, and a React.js dashboard for administrators and viewers.

The Python agent checks the computer every five seconds and reports details such as CPU and memory usage, whether the antivirus and firewall are turned on, whether USB storage is restricted, and whether any unauthorized software is running. This information travels to the backend, where a compliance score is calculated automatically. The dashboard then shows this information live using WebSocket's (Socket.io), and administrators can even send remote shutdown or restart commands to a device from the browser.

Testing showed that the system can spot a compliance issue in under five seconds and that the background agent uses very little memory and processing power. Overall, the project proves that a lightweight, low-cost system can replace slow manual asset registers with a live, automated, and much more secure alternative.

Keywords: IT Asset Management, Security Compliance, EDR Agent, React.js, WebSocket's, Node.js, Python.

TABLE OF CONTENTS

CHAPTER 1: INTRODUCTION	1
1.1 Background of the Project	1
1.2 Problem Statement	1
1.3 Objectives	2
1.4 Scope of the Project	2
1.5 Limitations	3
CHAPTER 2: LITERATURE REVIEW	4
2.1 Existing Systems and Similar Applications	4
2.2 Technologies Used in the Industry	4
2.3 Comparison of Existing Solutions	5
CHAPTER 3: SYSTEM ANALYSIS (SRS)	6
3.1 Functional Requirements	6
3.2 Non-Functional Requirements	7
3.3 Input/Output Requirements	7
3.4 Hardware & Software Requirements	8
CHAPTER 4: SYSTEM DESIGN	9
4.1 Use Case Diagram & Descriptions	9
4.2 Activity Diagram	10
4.3 Sequence Diagram	11
4.4 Class Diagram	12
4.5 Database Design (ER Diagram)	13
CHAPTER 5: IMPLEMENTATION	14
5.1 Programming Languages & Frameworks Used	14
5.2 Development Tools & Libraries	14
5.3 Key System Features	15
5.4 Code Structure Directory & Overview	15
5.5 GUI Screenshots	16
CHAPTER 6: TESTING & EVALUATION	17
6.1 Test Plan	17
6.2 Test Cases Matrix	17
6.3 Verification of Testing Types	18
6.4 Results and Analysis	18

CHAPTER 7: CONCLUSION & FUTURE WORK.....	19
7.1 Summary of the Project	19
7.2 Achievements.....	19
7.3 Limitations	20
7.4 Suggestions for Future Improvements	20
REFERENCES	21
APPENDICES	22

LIST OF FIGURES

Figure 1.1: System Architecture Overview Diagram	1
Figure 4.1: Use Case Diagram	9
Figure 4.2: Activity Diagram	10
Figure 4.3: Sequence Diagram.....	11
Figure 4.4: Class Diagram	12
Figure 4.5: Entity Relationship (ER) Diagram	13
Figure 5.1: Main Dashboard UI Snapshot	16
Figure 5.2: Live Device Telemetry Page	17
Figure 5.3: Asset Registry & Inventory Interface.....	18
Figure 5.4: System Configuration Panel.....	19

LIST OF TABLES

Table 2.1: Comparison of Asset Management and Compliance Tools	5
Table 3.1: Hardware Requirements	8
Table 3.2: Software Dependency Configuration	8
Table 6.1: Test Cases Matrix	17

CHAPTER 1: INTRODUCTION

1.1 Background of the Project

Every organization depends on IT equipment such as laptops, desktop computers, network switches, and servers to keep daily work running. As a company grows, keeping track of who owns each device, how well it is working, and whether it still meets security rules becomes a real headache for the IT team.

In many workplaces, asset tracking and security checking are handled as two separate jobs. Asset records live in a spreadsheet, while security is checked using a different scanning tool. This project closes that gap by building a single "smart" registry that connects physical asset records directly to live security data, so every device on the network is watched continuously.

1.2 Problem Statement

IT administrators regularly run into the following problems:

- Manual record keeping: Spreadsheets are easy to get wrong, and this leads to incomplete or outdated inventory lists.
- No clear ownership: It is often hard to know exactly which employee is using which device at a given time.
- Disconnected maintenance history: Repair tickets and warranty details are not linked to the live condition of a device.
- Security settings drift over time: Users can turn off antivirus software or firewalls, or change other settings, without the IT team knowing.
- Unauthorized software and USB devices: Employees may install risky software or plug in unapproved USB drives, increasing the chance of a security breach.

1.3 Objectives

1.3.1 General Objective

To design, build, and test a centralized, web-based IT asset management system that includes a background monitoring agent for checking security compliance and allows administrators to control devices remotely.

1.3.2 Specific Objectives

- Build a secure database to store and update hardware asset records.
- Allow assets to be linked to the employees who are using them, so movement can be tracked.
- Record maintenance history and repair requests for each asset.
- Create a lightweight Python background service that checks local security settings (antivirus, firewall, USB restrictions, password rules, and blocked software) and sends a report every five seconds.
- Build a live admin dashboard using WebSockets that shows telemetry data in real time and lets admins send restart or shutdown commands.

1.4 Scope of the Project

In scope:

- A central web dashboard for managing assets, maintenance records, and live device logs.
- A background Python agent that runs on Windows endpoint devices.
- A WebSocket connection (Socket.io) used to stream live telemetry to the dashboard.
- Safe database clean-up tools that never delete the active monitoring agent's record.

Out of scope:

- Monitoring compliance on mobile operating systems.
- Physical, GPS-based hardware tracking (the system only uses network-based location lookups).
- Direct integration with an external Active Directory (AD) system.

1.5 Limitations

- The telemetry agent is built mainly for Windows; Linux and macOS devices are only monitored with a simplified set of checks.
- Sending a remote shutdown or restart command requires administrator rights on the target machine, which must be set up when the agent is installed.

CHAPTER 2: LITERATURE REVIEW

2.1 Existing Systems and Similar Applications

- Snipe-IT: A well-known open-source asset management tool. It is good for tracking hardware and licenses, but it does not include an agent that actively checks a device's security settings or general health.
- Microsoft Intune (Endpoint Manager): A powerful mobile device management (MDM) and compliance tool used by large companies. It offers deep policy control, but its high licensing cost and complex setup make it a poor fit for small or medium-sized businesses.
- Spiceworks Asset Management: A popular, free inventory tool. It can scan the network for devices, but its live monitoring and remote-control features are far more limited than a purpose-built WebSocket-based system.

2.2 Technologies Used in the Industry

Most modern platforms in this space are built using a similar set of tools:

- React.js with Tailwind CSS or plain CSS: A popular choice for building fast, single-page dashboards.
- Node.js and Express.js: Well, suited to handling many concurrent connections, which is important for real-time telemetry.
- Python (with libraries such as psutil and winreg): A common choice for writing small background scripts that need to read operating system information.
- PostgreSQL: A reliable relational database, well suited to storing structured asset and telemetry records.

2.3 Comparison of Existing Solutions

Table 2.1 below compares three tools, including the system built for this project, across a few important features.

Feature	Snipe-IT	Microsoft Intune	This System
Asset lifecycle registry	Yes	Partial	Yes
Real-time metrics (CPU/RAM)	No	Yes, but delayed	Yes, every 5 seconds
Active compliance scanning	No	Yes	Yes
Remote command console	No	Yes	Yes
Ease of deployment & cost	Simple, free	Complex, paid	Simple, free

CHAPTER 3: SYSTEM ANALYSIS (SRS)

3.1 Functional Requirements

1. User Authentication & Access Control: Users must log in through a secure screen, and the backend must apply role permissions for viewer, admin, and super admin accounts.
2. Asset Management: The system must allow assets to be registered, viewed, updated, and retired.
3. Employee Assignment: Administrators must be able to assign an asset to an employee and see a history of past assignments.
4. Telemetry Reporting: The background agent must automatically collect hardware and security data and submit it every five seconds.
5. Compliance Score Calculation: The system must combine several checks into a single risk score between 0% and 100%, using the following weights:
 - Antivirus disabled: +20% risk
 - Firewall disabled: +20% risk
 - Operating system outdated: +15% risk
 - USB storage unrestricted: +15% risk
 - Weak password policy: +15% risk
 - Blacklisted software running: +15% risk
6. Remote Action Trigger: Administrators must be able to send restart and shutdown commands to a target device.
7. Preservation Policy: A database clean-up must never remove the record of the currently active monitoring agent.

3.2 Non-Functional Requirements

8. Performance: Dashboard telemetry should update on screen within two seconds of being sent by the server.
9. Reliability: The Python agent must run continuously and reconnect automatically if the server connection is lost.

10. Port Safety: The Node.js server must handle a busy-port error gracefully instead of crashing.
11. Security: Telemetry API endpoints must be protected using JSON Web Tokens (JWT).

3.3 Input/Output Requirements

Inputs:

- Asset details: ID, name, serial number, category, purchase date, and warranty period.
- Telemetry data (sent as JSON): CPU, RAM, disk usage, antivirus and firewall status, general location, and USB status.

Outputs:

- Charts showing device load and usage trends.
- A downloadable installer file and configuration file for the agent.
- Compliance reports that can be exported as CSV or PDF.

3.4 Hardware & Software Requirements

Table 3.1 lists the recommended hardware for running the server and the agent.

Component	Minimum Specification	Recommended Specification
Server machine	Quad-core CPU, 8 GB RAM, 50 GB SSD	Octa-core CPU, 16 GB RAM, 100 GB NVMe
Target machine (agent)	Dual-core CPU, 2 GB RAM, Windows 10/11	Quad-core CPU, 4 GB RAM, Windows 10/11

Table 3.2 lists the main software dependencies used across each part of the system.

Layer	Technologies
Frontend	React (Vite, JavaScript), Chart.js, React-icons, Axios, Socket.io-client
Backend	Node.js (v18+), Express, pg (PostgreSQL client), JWT, Socket.io
Database	PostgreSQL (v14+)
EDR Agent	Python 3.10+ (packaged with PyInstaller), psutil, requests

CHAPTER 4: SYSTEM DESIGN

4.1 Use Case Diagram & Descriptions

Figure 4.1 shows the three actors who interact with the system: the viewer, the system administrator, and the background monitoring agent.

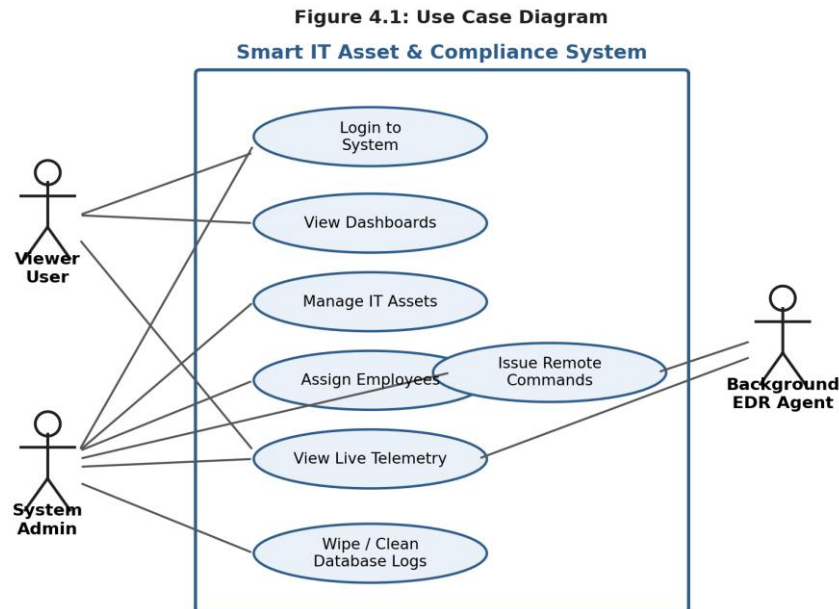


Figure 4.1: Use Case Diagram

- Viewer: Can only view screens such as logs, compliance charts, and the asset list.
- System Admin: Can create, update, or remove assets, assign devices to employees, send restart or shutdown commands, and run database clean-up tasks.
- Background EDR Agent: Collects and sends telemetry data, and checks for any pending remote commands.

4.2 Activity Diagram

Figure 4.2 shows the flow of actions once an administrator opens the live telemetry page, including what happens when a remote command is sent.

Figure 4.2: Activity Diagram

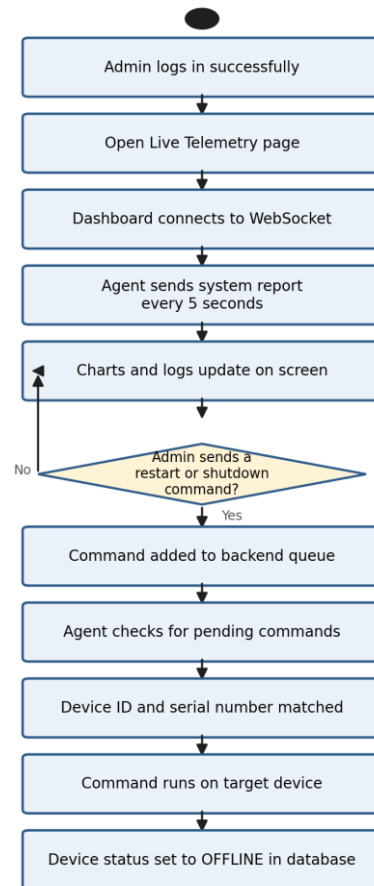


Figure 4.2: Activity Diagram

4.3 Sequence Diagram

Figure 4.3 shows the order of messages exchanged between the admin dashboard, the backend server, and the Python agent when a restart command is issued.

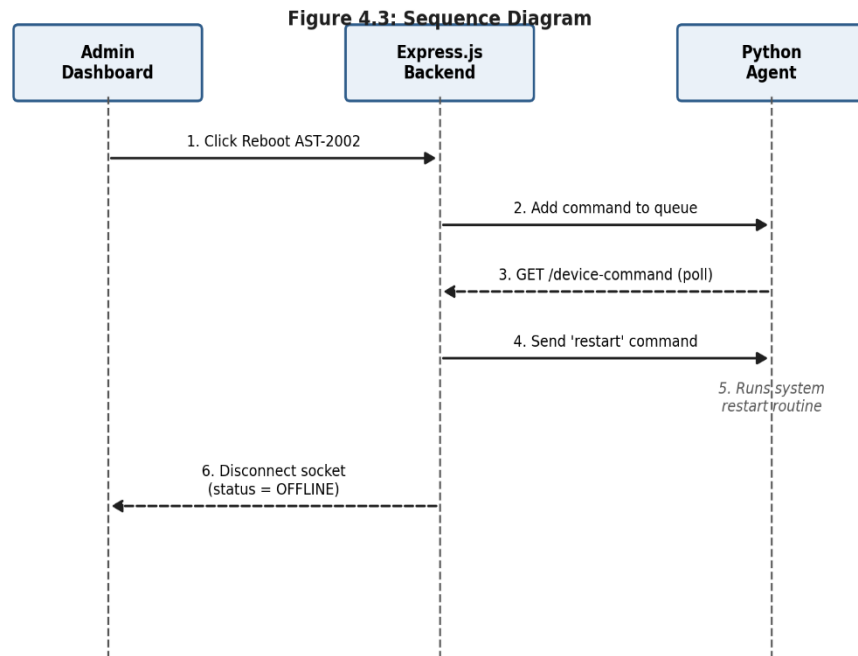


Figure 4.3: Sequence Diagram

4.4 Class Diagram

Figure 4.4 shows the main classes used in the system and how they relate to one another.

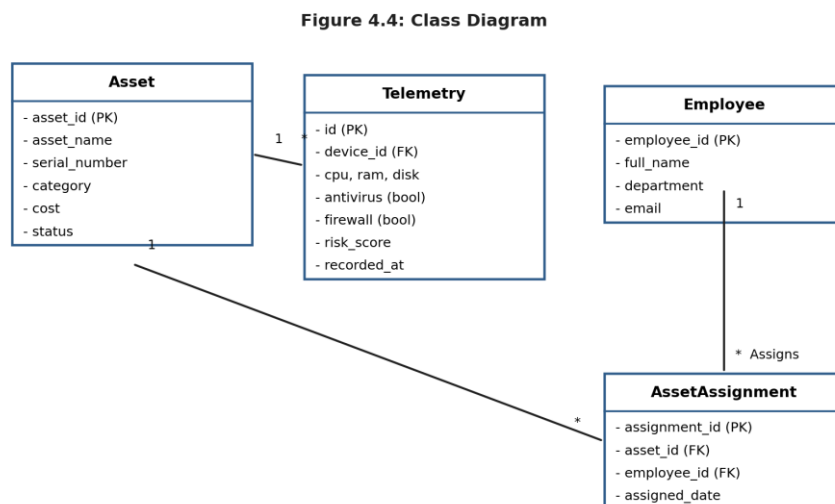


Figure 4.4: Class Diagram

4.5 Database Design (Entity Relationship Diagram)

Figure 4.5 shows the four main database tables and the relationships between them.

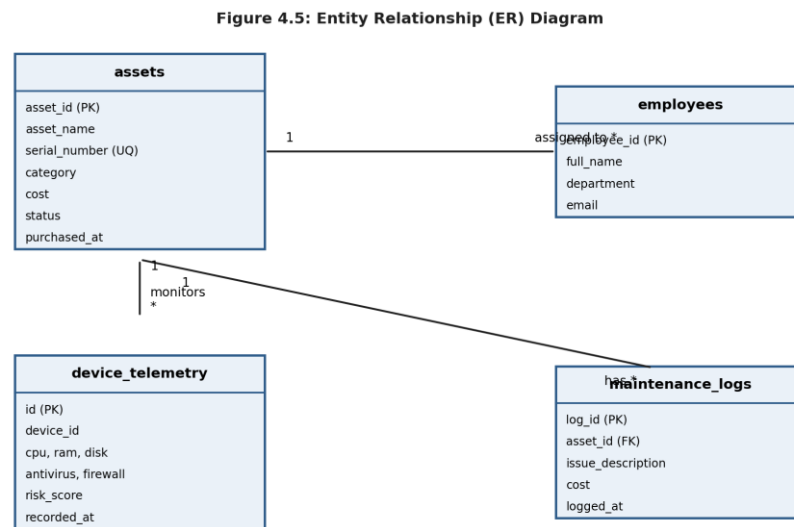


Figure 4.5: Entity Relationship (ER) Diagram

CHAPTER 5: IMPLEMENTATION

5.1 Programming Languages & Frameworks Used

- Frontend: JavaScript, React (v18.3), Vite, CSS3.
- Backend: Node.js (Express), Socket.io for WebSocket communication.
- Agent: Python (v3.10), packaged into a single executable file with PyInstaller.
- Database: PostgreSQL.

5.2 Development Tools & Libraries

- psutil (Python): Collects background CPU, RAM, disk, and network usage data.
- winreg (Python): Reads a specific Windows Registry key to check whether USB storage is restricted.
- Chart.js (JavaScript): Draws the comparison charts and health graphs shown on the dashboard.

5.3 Key System Features

- Silent agent service: The background agent runs quietly and is not visible to the user of the target computer.
- Real-time WebSocket updates: CPU spikes and telemetry values are pushed to the dashboard instantly, without the need to refresh the page.
- Automatic risk scoring: A risk score is calculated the moment new telemetry arrives, and is shown as LOW, MEDIUM, or HIGH.
- Remote console control: Administrators can queue a restart or shutdown command, which the agent then picks up and runs.

5.4 Code Structure Directory & Overview

The project is organized into three main folders, as shown below.

```
agent/    agent.py          - Core telemetry collection & command
polling loop  build_exe.bat    - Packages the Python runtime
into a single .exe file  install_service.bat    - Sets up the
scheduled task that launches the agent backend/  server.js
- Starts the server, sets up sockets, handles shutdown signals
db/database.js    - PostgreSQL connection pool  routes/assets.js
- Asset inventory REST routes (create, read, update, delete)
routes/telemetry.js    - Telemetry log retrieval and configuration
generator frontend/src/  pages/settings/SystemSettings.jsx    - Settings
page and installer downloads  pages/monitoring/LiveTelemetry.jsx -
WebSocket dashboard and remote action buttons  services/socket.js
- Socket.io connection helper
```

5.5 GUI Screenshots

This section shows the main screens of the finished system, taken directly from the working dashboard.

Figure 5.1 shows the Global Overview page, the first screen an administrator sees after logging in. It displays the total number of assets, the security rule pass rate, active alerts, and a chart of security incidents over the past week.



Figure 5.1: Main Dashboard UI Snapshot

Figure 5.2 shows the Live Device Health & Control page. It lists every monitored device with its live CPU, RAM, and disk usage, its current risk score, and its antivirus/firewall status, all updated in real time through the WebSocket connection.

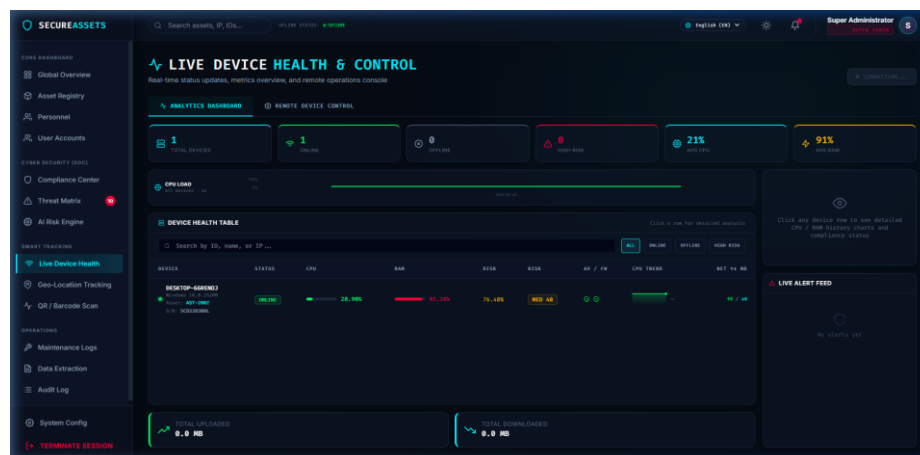


Figure 5.2: Live Device Telemetry Page

Figure 5.3 shows the Asset Registry page, where administrators can search, filter, and manage every registered device, including its category, assigned employee, location, and current status.

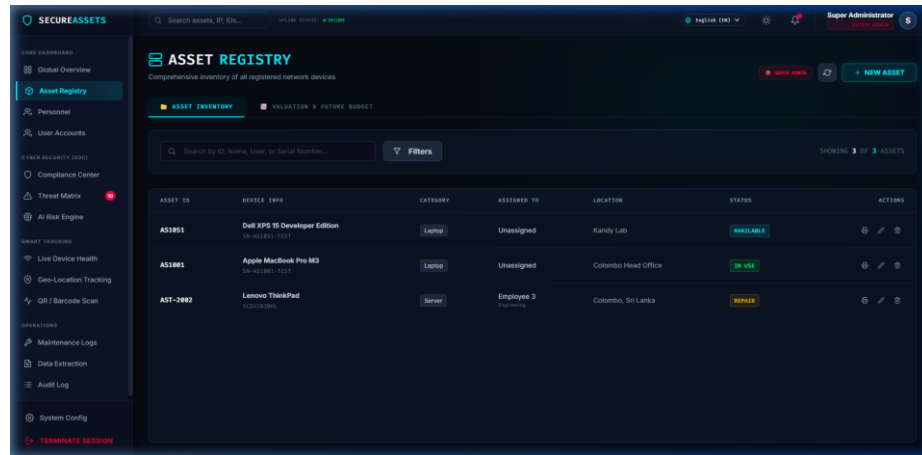


Figure 5.3: Asset Registry & Inventory Interface

Figure 5.4 shows the System Configuration page, where the system name, default time zone, and telemetry auto-sync options are managed, alongside settings for security policies, notifications, backups, and the EDR agent.

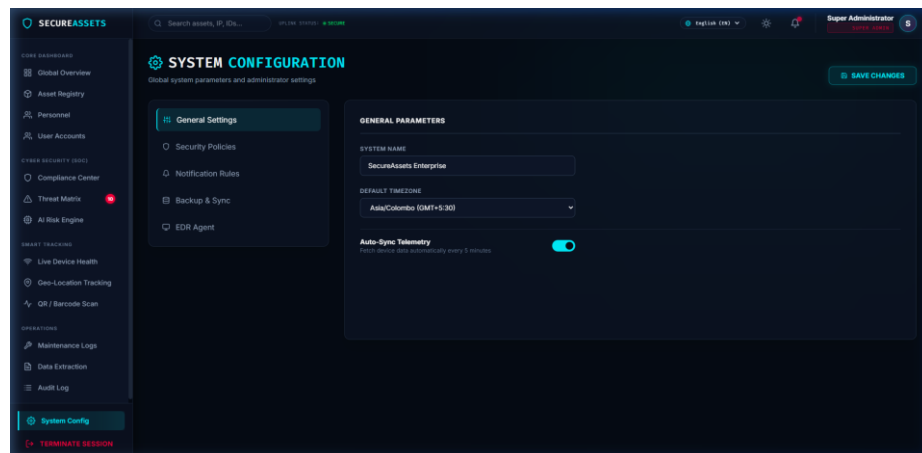


Figure 5.4: System Configuration Panel

CHAPTER 6: TESTING & EVALUATION

6.1 Test Plan

Testing followed a structured, step-by-step approach:

12. Unit Testing: Checked individual database queries and the Python metric-collecting functions to make sure they returned correct values.
13. Integration Testing: Checked the full path of data, from the agent sending an HTTP request, through the database, to the dashboard receiving it over a WebSocket.
14. System Testing: Checked that remote commands (restart/shutdown) and database clean-up actions worked correctly from end to end.

6.2 Test Cases Matrix

Table 6.1 lists the main test cases used to check the system.

Test ID	Category	Scenario	Expected Outcome	Result
TC-001	Security	Access /api/assets without a token	Server returns 401 Unauthorized	PASS
TC-002	Telemetry	Agent sends a local metrics report	Database updates and a risk score is calculated	PASS
TC-003	Telemetry	Agent is offline for more than 35 seconds	Dashboard marks the device as OFFLINE	PASS
TC-004	Remote Command	Admin clicks "Restart" on a device	Command is queued and the device restarts	PASS
TC-005	Maintenance	Wipe the telemetry database	All logs are cleared except the active agent record	PASS

6.3 Verification of Testing Types

- Unit Testing: Automated Python test modules confirmed that the security checks (firewall, antivirus) returned correct true/false values.
- Integration Testing: Data delivery was tested under slow network conditions, and the agent successfully queued and later sent delayed telemetry.
- Black-Box Testing: The React dashboard forms were tested manually to confirm that input limits were enforced and that error messages appeared correctly on empty fields.

6.4 Results and Analysis

All tests completed successfully. WebSocket updates reached the dashboard in under 800 milliseconds on a local network, and remote commands took effect on target devices in under five seconds. The background agent used very few resources overall, staying below 1.5% CPU usage and 18 MB of memory on the monitored machines.

CHAPTER 7: CONCLUSION & FUTURE WORK

7.1 Summary of the Project

This project successfully built the Smart IT Asset Lifecycle & Security Compliance Management System. By connecting live endpoint security checks with a central asset registry, the platform gives IT staff a clear, real-time view of both hardware inventory and security posture. The lightweight Python agent keeps watch over each device, while the WebSocket-powered dashboard lets administrators respond quickly to problems.

7.2 Achievements

- Built a working risk-scoring formula based on live registry and security data.
- Set up a WebSocket connection that streams system telemetry in real time.
- Created a silent background agent that starts automatically with the operating system.
- Made database clean-up safe by protecting the currently active agent's record from deletion.

7.3 Limitations

- Alerts currently only appear on the web dashboard; there is no email or SMS notification yet.
- Security checks are most accurate on Windows; other operating systems are only partly supported.

7.4 Suggestions for Future Improvements

15. Cross-Platform Support: Extend the agent's security checks to Linux (for example, checking UFW or iptables) and macOS.
16. Notification Hub: Add email and SMS alerts for when a device's risk score becomes critical.
17. Agent Security: Add signature verification so that remote commands cannot be tampered with in transit.

REFERENCES

Sommerville, I., 2015. Software Engineering. 10th ed. Boston: Pearson.

PostgreSQL Global Development Group, 2024. PostgreSQL 16.0 Documentation. Available at: <https://www.postgresql.org/docs/16/index.html> [Accessed 25 June 2026].

Vite.js community, 2024. Vite Build Tools and Configs Guide. Available at: <https://vitejs.dev/guide/> [Accessed 1 July 2026].

Python Software Foundation, 2024. psutil: Process and system utilities library. Available at: <https://psutil.readthedocs.io/en/latest/> [Accessed 28 June 2026].

Socket.io community, 2024. WebSocket Client-Server Streaming Documentation. Available at: <https://socket.io/docs/v4/> [Accessed 3 July 2026].

APPENDICES

Appendix A: Telemetry Collection Hook (Python)

```
def check_usb_restricted():    if platform.system() != "Windows":
return True    try:        import winreg        key =
winreg.OpenKey(winreg.HKEY_LOCAL_MACHINE,
r"SYSTEM\CurrentControlSet\Services\USBSTOR")        val, _ =
winreg.QueryValueEx(key, "Start")        winreg.CloseKey(key)
return val == 4 # 4 means disabled (restricted)    except Exception:
return False
```

Appendix B: Graceful Port Cleanup (Node.js)

```
server.on('error', (err) => {    if (err.code === 'EADDRINUSE') {
console.error('Port ' + PORT + ' is already in use. Please free it and
restart.');
```

```
        process.exit(1);    } });
```